NARESH EDAGOTTI

FastAPI Cheat Sheet

Everything an AI engineer actually ships, from first route to production.

Why FastAPI Wins for AI Work

AI endpoints are slow, IO bound and return structured data. That combination is exactly what FastAPI was built for.

ASYNC	One worker handles hundreds of waiting model calls
PYDANTIC	The same models validate requests and LLM output
STREAMING	Token by token responses without extra libraries
AUTO DOCS	OpenAPI and a live test UI generated from your types

main.py

```python
from fastapi import FastAPI

app = FastAPI(title="AI Service", version="1.0")

@app.get("/health")
async def health():
    return {"status": "ok"}
```

Visit /docs on any running app. That interactive page is generated free from your type hints.

NARESH EDAGOTTI

Install, Run, Structure

terminal

```
# everything you need, one install
pip install "fastapi[standard]"

# dev server with hot reload
fastapi dev app/main.py

# production, explicit and boring
uvicorn app.main:app --host 0.0.0.0 --port 8000
```

LAYOUT THAT SCALES PAST ONE FILE

project tree

```
app/
  main.py         # app object, router includes
  config.py       # settings from env
  deps.py         # shared dependencies
  schemas.py      # pydantic request and response
  routers/
    chat.py       # APIRouter per domain
    documents.py
  services/
    llm.py        # model clients, no FastAPI here
```

Keep model code out of routers. Routers handle HTTP, services handle intelligence, and each stays testable alone.

NARESH EDAGOTTI

One Kit to Clear Your AI Interview

After seeing how AI interviews are evolving, I built an **AI Interview Kit** around the areas companies actually test.

12	0	1
CORE TOPICS	RANDOM DUMPS	STRUCTURED PATH

01	AI fundamentals	07	Prompt engineering
02	AI agents	08	Backend for AI
03	Transformers in depth	09	Fine tuning
04	Memory systems	10	Deployment
05	LLM architectures	11	RAG systems
06	Databases	12	GenAI system design

Everything structured to help you prepare

Check the caption to enroll

NARESH EDAGOTTI

Reading the Request

FastAPI decides where a parameter comes from by its type. Scalars in the path or query, Pydantic models in the body. No parsing code, ever.

routers/chat.py

```python
from fastapi import APIRouter, Query, Path, Header

router = APIRouter(prefix="/chat", tags=["chat"])

@router.post("/{session_id}/messages")
async def send(
    session_id: str = Path(min_length=8),
    body: ChatRequest = ...,                # json body
    top_k: int = Query(4, ge=1, le=20),     # ?top_k=4
    x_request_id: str | None = Header(None),
):
    return await answer(session_id, body.message, top_k)
```

PATH	Identifies a resource, always required
QUERY	Filters and options, give every one a default
BODY	Anything structured, validated by Pydantic

NARESH EDAGOTTI

Schemas and response_model

Define the contract once. The same class validates input, shapes output, and writes your API docs.

schemas.py

```python
from pydantic import BaseModel, Field, field_validator

class ChatRequest(BaseModel):
    message: str = Field(min_length=1, max_length=4000)
    temperature: float = Field(0.0, ge=0, le=2)

    @field_validator("message")
    def no_blank(cls, v):
        if not v.strip(): raise ValueError("empty")
        return v.strip()

class ChatResponse(BaseModel):
    reply: str
    sources: list[str] = []
    tokens_used: int
```

routers/chat.py

```python
@router.post("/", response_model=ChatResponse, status_code=201)
async def chat(req: ChatRequest): ...
```

response_model filters the output. Extra fields your service returns are stripped, so an internal key can never leak by accident.

NARESH EDAGOTTI

async def or def

The single mistake that kills AI services in production. **async def** runs on the event loop. One blocking call inside it freezes every other request on that worker.

BLOCKS EVERYTHING

async def with a sync SDK call, or a torch inference, or time.sleep

SAFE

plain def, FastAPI runs it in a threadpool automatically

routers/chat.py

```python
# WRONG: sync client inside async, loop is stuck
@router.post("/bad")
async def bad(req: ChatRequest):
    return client.chat.completions.create(...)

# RIGHT 1: async client, await it
@router.post("/good")
async def good(req: ChatRequest):
    return await async_client.chat.completions.create(...)

# RIGHT 2: CPU bound, use plain def
@router.post("/embed")
def embed(req: EmbedRequest):
    return model.encode(req.texts)   # threadpool
```

⚠ Stuck with a sync client in async code? Wrap it: await run_in_threadpool(fn, args).

NARESH EDAGOTTI

AI SPECIFIC

PRACTICAI

Load the Model Once

Loading a model inside a request handler is the most expensive bug in AI backends. Lifespan runs on startup, holds the object, and cleans up on shutdown.

main.py

```python
from contextlib import asynccontextmanager

@asynccontextmanager
async def lifespan(app: FastAPI):
    # startup: runs once per worker process
    app.state.embedder = SentenceTransformer(MODEL_NAME)
    app.state.store = await open_vector_store()
    app.state.http = httpx.AsyncClient(timeout=30)
    yield
    # shutdown: release everything you opened
    await app.state.http.aclose()

app = FastAPI(lifespan=lifespan)
```

LOAD HERE	Embedding models, tokenizers, DB pools, HTTP clients
NEVER HERE	Anything per user, state must stay request scoped
MEMORY	Each worker loads its own copy, so 4 workers is 4x RAM

NARESH EDAGOTTI

CORE PATTERN

Dependency Injection

Depends is FastAPI's best idea. Declare what an endpoint needs and the framework supplies it, resolves nested dependencies, and lets tests swap any of it out.

deps.py

```python
from fastapi import Depends, Request
from typing import Annotated

def get_store(request: Request):
    return request.app.state.store

async def get_db():
    async with SessionLocal() as s:
        yield s          # cleanup runs after response

Store = Annotated[VectorStore, Depends(get_store)]
Db    = Annotated[AsyncSession, Depends(get_db)]

# now endpoints read like plain functions
@router.post("/search")
async def search(q: str, store: Store, db: Db): ...
```

In tests, app.dependency_overrides[get_store] = fake_store swaps the real vector DB for a stub with one line.

NARESH EDAGOTTI

Stream Tokens with SSE

Users judge an LLM app on time to first token, not total time. Server sent events give you streaming over plain HTTP with no extra dependency.

routers/chat.py

```python
from fastapi.responses import StreamingResponse
import json

async def token_stream(prompt: str):
    try:
        async for chunk in llm.astream(prompt):
            payload = json.dumps({"delta": chunk.content})
            yield f"data: {payload}\n\n"
        yield "data: [DONE]\n\n"
    except Exception as e:
        yield f'data: {{"error":"{e}"}}\n\n'

@router.post("/stream")
async def stream(req: ChatRequest):
    return StreamingResponse(
        token_stream(req.message),
        media_type="text/event-stream",
        headers={"X-Accel-Buffering": "no"})
```

⚠ Errors after the first byte cannot become a 500. Yield an error event instead, and never let an exception escape the generator.

NARESH EDAGOTTI

WebSockets for Chat

SSE is one way. When the client also needs to talk mid stream, to interrupt a generation or send a tool result, use a socket.

routers/ws.py

```python
from fastapi import WebSocket, WebSocketDisconnect

@app.websocket("/ws/{session_id}")
async def chat_ws(ws: WebSocket, session_id: str):
    await ws.accept()
    history = []
    try:
        while True:
            msg = await ws.receive_json()
            history.append(msg["text"])
            async for chunk in llm.astream(history):
                await ws.send_json({"delta": chunk.content})
            await ws.send_json({"done": True})
    except WebSocketDisconnect:
        await save_session(session_id, history)
```

PICK SSE	One way token streaming, simpler and proxy friendly
PICK WS	Interrupts, live tool calls, collaborative sessions

NARESH EDAGOTTI

File Uploads for RAG

Every RAG product needs an ingest endpoint. UploadFile streams to disk instead of loading the whole PDF into memory.

routers/documents.py

```python
from fastapi import UploadFile, File, HTTPException

MAX_MB, OK_TYPES = 25, {"application/pdf", "text/plain"}

@router.post("/ingest")
async def ingest(files: list[UploadFile] = File(...)):
    for f in files:
        if f.content_type not in OK_TYPES:
            raise HTTPException(415, f"bad type
{f.content_type}")

        size = 0
        while block := await f.read(1 << 20):   # 1MB chunks
            size += len(block)
            if size > MAX_MB * 1_000_000:
                raise HTTPException(413, "file too large")
            await write_chunk(f.filename, block)
    return {"ingested": len(files)}
```

Never trust content_type or filename, both come from the client. Sniff the real magic bytes and generate your own storage name.

Background Tasks

Return a response now, finish the work after. Perfect for logging and cleanup, and completely wrong for anything you cannot afford to lose.

routers/documents.py

```python
from fastapi import BackgroundTasks

@router.post("/documents", status_code=202)
async def upload(file: UploadFile, bg: BackgroundTasks):
    doc_id = await save_raw(file)
    bg.add_task(embed_and_index, doc_id)   # after response
    return {"doc_id": doc_id, "status": "processing"}
```

BACKGROUNDTASKS

Short, cheap, safe to lose. Emails, audit logs, cache warming

NEEDS A QUEUE

Long embedding jobs, retries, anything a restart must not drop

⚠ Background tasks live inside the worker process. A deploy, crash or scale down loses them silently, with no retry.

NARESH EDAGOTTI

Errors That Make Sense

Model providers time out, rate limit and return garbage. Map every failure to a status code the client can act on.

main.py

```python
from fastapi import HTTPException, Request
from fastapi.responses import JSONResponse

class ModelUnavailable(Exception): ...

@app.exception_handler(ModelUnavailable)
async def on_model_down(req: Request, exc):
    return JSONResponse(
        status_code=503,
        content={"error": "model_unavailable",
                 "retry_after": 30},
        headers={"Retry-After": "30"})

# inline, for the ordinary cases
if not doc:
    raise HTTPException(404, "document not found")
```

422	Pydantic rejected the request, automatic
429	Your rate limit, or the provider's, was hit
503	Model down, always send Retry-After
504	Provider exceeded your timeout budget

Middleware and CORS

Middleware wraps every request. Use it for the three things you need on day one: browser access, request tracing and latency visibility.

main.py

```python
from fastapi.middleware.cors import CORSMiddleware
import time, uuid

app.add_middleware(
    CORSMiddleware,
    allow_origins=settings.allowed_origins,  # never ["*"]
    allow_methods=["GET", "POST"],
    allow_headers=["*"])

@app.middleware("http")
async def observe(request: Request, call_next):
    rid = request.headers.get("X-Request-ID", str(uuid.uuid4()))
    start = time.perf_counter()
    response = await call_next(request)
    ms = (time.perf_counter() - start) * 1000
    response.headers["X-Request-ID"] = rid
    response.headers["X-Process-Time"] = f"{ms:.1f}ms"
    return response
```

Put the request id into your log context and pass it to the LLM trace. One id then links an HTTP call to its exact model run.

NARESH EDAGOTTI

API Keys and Bearer Tokens

Auth is just a dependency. Attach it to a router and every route under it is protected, including ones you add later.

deps.py

```python
from fastapi.security import APIKeyHeader, HTTPBearer
import secrets

api_key_header = APIKeyHeader(name="X-API-Key")

async def require_key(key: str = Security(api_key_header)):
    # constant time compare, avoids timing leaks
    if not secrets.compare_digest(key, settings.api_key):
        raise HTTPException(401, "invalid api key")
    return key

# protect a whole router in one line
router = APIRouter(
    prefix="/v1",
    dependencies=[Depends(require_key)])
```

API KEY	Service to service, rotate it, scope it per client
BEARER JWT	End users, carries identity and expiry
ALWAYS	Compare with compare_digest, never with ==

NARESH EDAGOTTI

Concurrency and Timeouts

A GPU serves a fixed number of requests at once. Without a limit, a traffic spike does not slow your service down, it takes it out entirely.

services/llm.py

```python
import asyncio

# hard cap on simultaneous model calls
gpu_slots = asyncio.Semaphore(4)

async def generate(prompt: str):
    try:
        async with asyncio.timeout(30):
            async with gpu_slots:
                return await model.agenerate(prompt)
    except TimeoutError:
        raise HTTPException(504, "generation timed out")

# and cap the server itself
# uvicorn main:app --limit-concurrency 100 \
#   --timeout-keep-alive 5
```

SEMAPHORE	Protects the scarce resource, usually GPU or a paid quota
TIMEOUT	Always set one, providers can hang for minutes
SHED LOAD	Return 429 fast, far kinder than queueing forever

NARESH EDAGOTTI

Config and Testing

config.py

```python
from pydantic_settings import BaseSettings

class Settings(BaseSettings):
    api_key: str
    model_name: str = "gpt-4o-mini"
    max_concurrency: int = 4
    class Config: env_file = ".env"

settings = Settings()   # fails loudly at boot if unset
```

tests/test_chat.py

```python
from fastapi.testclient import TestClient

app.dependency_overrides[get_llm] = lambda: FakeLLM()
client = TestClient(app)

def test_chat_returns_sources():
    r = client.post("/v1/chat/", json={"message": "hi"},
                    headers={"X-API-Key": "test"})
    assert r.status_code == 201
    assert r.json()["sources"]
```

Override the model dependency with a fake. Your API tests then run in milliseconds, cost nothing, and never flake on a provider outage.

NARESH EDAGOTTI

Shipping It

Dockerfile

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app ./app
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", \
     "--port", "8000", "--workers", "2"]
```

main.py

```
@app.get("/health")      # is the process alive
async def health(): return {"ok": True}

@app.get("/ready")       # is the model actually loaded
async def ready(request: Request):
    if not getattr(request.app.state, "embedder", None):
        raise HTTPException(503, "model not loaded")
    return {"ready": True}
```

WORKERS	CPU work scales with cores, GPU work usually stays at one
TWO PROBES	Health for liveness, ready for traffic, never the same route
STARTUP	Model loading is slow, raise the readiness grace period

NARESH EDAGOTTI

RECAP

PRACTICAI

The AI Backend Checklist

`async def or def`	Never block the event loop
`lifespan`	Load models once, not per request
`Depends`	Inject clients, swap them in tests
`response_model`	Shape output, stop field leaks
`StreamingResponse`	Time to first token is the metric
`Semaphore`	Cap concurrent GPU work
`Timeouts`	On every provider call, no exceptions
`health and ready`	Two probes, two different questions

The model is the easy part.
The API around it is
what makes it a product.

Save this for your next AI service. Follow for more.

NARESH EDAGOTTI